CUET PG

DAILY CLASS NOTES

Computer Science

Digital Fundamental

Lecture – 08

Boolean Algebra and Logic Gates: Part-1



Boolean Algebra and Logic Gates: Part-1

Content:
Axioms of Boolean Algebra, Some Useful Theorems, Unique Features, Boolean Subalgebra, Basic Operations, Logic Gates.

Definition of Boolean Algebra:

- A Boolean algebra is a non-empty set B with operations + , . , ' and constants 0 and 1 satisfying certain rules.

Axioms of Boolean Algebra:

- If a, b, c ∈ B
- Commutative Laws
  - a + b = b + a
  - a · b = b · a
- Distributive Laws
  - a + (b · c) = (a + b)(a + c)
  - a · (b + c) = (a · b) + (a · c)
- Identity Laws
  - a + 0 = a
  - a · 1 = a
- Complement Laws
  - a + a' = 1
  - a · a' = 0

Basic Theorems

- Idempotent Laws:
  - a + a = a
  - a · a = a
- Boundedness Laws:
  - a + 1 = 1
  - a · 0 = 0
- Absorption Laws:
  - a + (a · b) = a
  - a(a + b) = a
- Associative Laws:
  - (a + b) + c = a + (b + c)
  - (a · b) · c = a · (b · c)
- Uniqueness of Complement:
  - If a + x = 1 and a·x = 0 → x = a'
- Involution Law:
  - (a')' = a
  - 0' = 1
  - 1' = 0

- De-Morgan's Laws:
  - (a + b)' = a'b'
  - (ab)' = a' + b'
- When it is clear from context, we can remove the dot (·) symbol in Boolean algebra.
  Example: Instead of writing a · b, we can simply write ab.
  This is allowed only when there is no chance of misunderstanding.
- Some Boolean laws have been shown with algebraic proofs in the lesson.
- The remaining laws are given as homework or practice for the learner to prove on their own.
- **Point to Remember:**
  - Each of the laws can also be proved using a truth table.
  - Theorem : a + a =a
  - Theorem: a.a =a
  - Theorem: a +1 =1
  - Theorem : a+ (a.b) =a
  - Theorem: for each a ∈ B, a' is unique.
  - Theorem: The element 0 and 1 are unique
  - Theorem: (i) (a+b)' = a'b' (ii) (a.b)' = a' + b'

Unique Features of Boolean Algebra:

- Only two values: 0 and 1
  - Boolean algebra uses only 0 (false) and 1 (true).
  - Ordinary algebra uses infinite values (1,2,3,… etc.).
  - So, every Boolean variable can have only two possible values: 0 or 1.
- No subtraction or division
  - Boolean algebra supports only AND, OR, NOT operations.
  - It does not have subtraction (−) or division (÷) like normal algebra.
- Complement (NOT) operation is unique
  - Boolean algebra has a special unary operation called complement (A').
  - Ordinary algebra has no equivalent of this bar/NOT operation.

  Boolean Subalgebra:
  - A Boolean subalgebra is a subset of a Boolean algebra.
  - This subset must be non-empty.
  - It is called a subalgebra only if it follows the same operations:
    OR (+)
    AND (·)
    NOT (')
    and contains 0 and 1
- Meaning:
  - If a, b are in A → then a + b, a·b, and a' must also be in A.
  - Example Set (D_{30})

  Consider the set of all divisors of 30:
  D_{30} = {1, 2, 3, 5, 6, 10, 15, 30}

- Operations defined on D_{30}
  - $a + b = \text{lcm}(a, b)$ (Where + means LCM, not normal addition)
  - $a \cdot b = \text{gcd}(a, b)$ (Where · means GCD, not multiplication)
  - $a' = 30 / a$ (Complement is defined as 30 divided by the number)
- D_{30} is a Boolean algebra
  - With the operations defined earlier (LCM, GCD, complement),
  - D_{30} becomes a valid Boolean algebra.

  Examples of Boolean Subalgebras of D_{30}

  $A_1 = \{1, 5, 6, 30\}$

  $A_3 = \{1, 2, 3, 6\}$

  These sets are Boolean subalgebras because:

  They are closed under the operations:

  $a + b = \text{LCM}(a, b)$

  $a \cdot b = \text{GCD}(a, b)$

  $a' = 30/a$

  Meaning: Doing any of these operations on elements of A_1 or A_3 produces a result that still stays inside the same set.
- A_4 is NOT a Boolean Subalgebra

  $A_4 = \{1, 2, 3, 6\}$

  When we take the complement of 6:

  $6' = 30/6 = 5$

  But 5 is not in A_4, so A_4 is not closed under complement.

  Therefore, A_4 is not a Boolean subalgebra.
- Checking Closure Rule (Important Point)

  To prove a set A is a Boolean subalgebra, you do not need to check all operations.

  It is enough to check closure under:

  {+, ’} (LCM and complement)

  OR

  {·, ’} (GCD and complement)

  If these two operations keep results inside A, it is a subalgebra.

Basic Operations:

- Three operations:
  - Logical Addition (OR)
  - Logical Multiplication (AND)
  - Complementation (NOT)

Logic 0	**False**	**Off**	**Low**	**Open**	**No**
Logic 1	**True**	**On**	**High**	**Closed**	**Yes**

- Logical Addition(OR):
  - Each variable in Boolean algebra has either two values: true or false (1 or 0), the operation can be defined as follows:
  - Truth values:

    $0 + 0 = 0$ $0 + 1 = 1$ $1 + 0 = 1$ $1 + 1 = 1$
  - '+' is logical OR (not arithmetic plus).

❖ Truth Table:[Output is 1 when at least one input is 1.]

A	B	A + B (OR)
0	0	0
0	1	1
1	0	1
1	1	1

❖ Logical Multiplication(AND):

★ Logical multiplication means the AND operation in Boolean algebra.

★ Symbol used:

$A \cdot B$ (dot)

or simply AB

or written as A AND B

★ AND Rule:

Output is 1 only when both inputs are 1; otherwise, output is 0.

★ Truth Values:

$0 \cdot 0 = 0$ $0 \cdot 1 = 0$ $1 \cdot 0 = 0$ $1 \cdot 1 = 1$

❖ Truth Tables:[output is 1 only when both inputs are 1.]

A	B	A. B (AND)
0	0	0
0	1	0
1	0	0
1	1	1

❖ Complementation(NOT):

★ In Boolean Algebra, *complementation* is an operation represented by a **bar (–)** over a variable or an **apostrophe (′)** symbol.

Symbol Representation:

$0'=1$

$1'=0$

Interpretation:

A′ means the **complement of A**.

It is read as **"NOT A"**.

The complement gives the **opposite logic value** of A.

If A=1, then A′=0

If A=0, then A′=1.

★ **Duality Principle**

1. **Definition:**

   The **duality principle** in Boolean algebra states that if an expression or theorem is true, then its **dual**—obtained by interchanging certain operations and identity elements—is also true.

2. **Interchange Rules:**

   - Replace every **AND (·)** with **OR (+)**.
   - Replace every **OR (+)** with **AND (·)**.
   - Replace every **0** with **1**.
   - Replace every **1** with **0**.

3. **Key Idea:**
   - This property of changing operations and identity elements is what defines *duality*.
   - If a theorem is proved using Boolean algebra axioms, its dual theorem **automatically holds true**.
4. **Axioms and Theorems:**
   - The axioms and basic theorems of Boolean algebra are often grouped in **pairs (a)** and **(b)**, where one can be obtained from the other by applying the duality principle.
5. **Example:**
   - Given: $f=(a\cdot 1)\cdot(0\cdot a')$
   - Dual: $f^D = (a + 0) + (1 + a')$
   - This dual is obtained by interchanging **AND (·) ↔ OR (+)** and **0 ↔ 1**.

★ **De Morgan's Theorem**

1. **Meaning:**
   - De Morgan's Theorem is used to find the **complement** of any Boolean expression.
   - It helps in simplifying and transforming logic expressions.
2. **Rule:**

   There are **two parts** to De Morgan's Theorem:
   - $(x+y)'=x' \cdot y'$ → The complement of an OR equals the AND of complements.
   - $(xy)'=x'+y'$ → The complement of an AND equals the OR of complements.
3. **Usage:**
   - Apply these theorems **step by step** to find the complement of larger Boolean expressions.
   - They are especially useful in **logic circuit simplification** and **digital design**.

x	y	x'	y'	x+y	(x+y)'	x'y'	xy	(xy)'	x'+y'
0	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	0	0	1	1
1	0	0	1	1	0	0	0	1	1
1	1	0	0	1	0	0	1	0	0

Extended De Morgan's Theorem :

1. **Extension to Multiple Variables:**
   - De Morgan's Theorems can be applied to **more than two variables**.
2. **Example 1 – For OR Operation:**

   $(x+y+z)'=x'\,y'\,z'$
   - Hence, the complement of an OR of multiple variables equals the **AND of their complements**.
3. **Example 2 – For AND Operation:**

   $(x\,y\,z)'=x'+y'+z'$
   - Hence, the complement of an AND of multiple variables equals the **OR of their complements**.
4. **Generalization:**
   - These results can be **extended to any number of variables** using the same pattern.
5. **Applicability:**
   - The theorems are **equally valid** even if **x and/or y** represent **complex Boolean expressions** (containing more than one variable).

Combined De Morgan's Theorem:

1. **Purpose:**
   - The two forms of De Morgan's Theorem can be combined into **one general rule** to find the **complement of any Boolean expression**.
2. **Steps to Obtain the Complement:**
   Replace the following in the expression:
   (a) **0 → 1**
   (b) **1 → 0**
   (c) **+ → · (AND)**
   (d) **· → + (OR)**
   (e) **Each variable → Its complement**
3. **Generalized Formula:**
   $[f(X1,X2,\ldots,Xn,0,1,+,\cdot)]' = f(X1',X2',\ldots,Xn',1,0,\cdot,+)$
   Explanation:
   - This rule helps to **systematically find the complement** of any Boolean function, no matter how complex.
   - It combines the logic of both De Morgan's theorems into a **single transformation rule**.

Logic Gate :

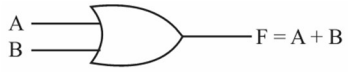
1. **Definition:**
   - A **logic gate** is an **electronic circuit** that performs a specific logical operation on one or more **input signals** to produce a **single output signal**.
2. **Nature:**
   - Logic gates are **digital circuits**, also called **logic circuits**, because they work with **binary signals (0 and 1)**.
3. **Representation:**
   - Each gate is represented by a **symbol** in circuit diagrams.
   - **Input lines** enter the symbol from the **left side**.
   - The **output line** appears on the **right side**.
4. **Complement Signal:**
   - A **small circle (bubble)** placed on an input or output line represents a **complement (inversion)** of the signal on that line.
5. **Purpose:**
   - Logic gates form the **basic building blocks** of digital electronics and are used in **computers, processors, and logic-based devices**.

OR Gate:

1. **Definition:**
   - An **OR gate** is an electronic circuit that receives **two or more input signals** and produces an **output** that is the **logical OR** of the inputs.
2. **Logic Function:**
   - The output is **1 (High)** if **at least one input** is **1 (High)**.
   - The output is **0 (Low)** only when **all inputs are 0 (Low)**.
3. **Input States:**
   - Each input signal has **two stable states**:
     - **1 (High)** → Logic True
     - **0 (Low)** → Logic False
4. **Symbol:**
   - The OR gate symbol shows multiple input lines on the **left**, joined into a **curved shape**, with a **single output line** on the **right**.

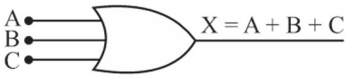
5. **Truth Table (for 2-input OR Gate):**

Input A	Input B	Output (A + B)
0	0	0
0	1	1
1	0	1
1	1	1



Symbol for an OR gate

The same idea can be extended to more than two inputs. A three-input OR gate and its truth table.



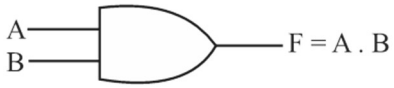
Symbol for these input OR gate

Input			Output
A	**B**	**C**	**X = A + B + C**
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

AND gate-

★ An AND gate is an electronic device that accepts the values of two or more Boolean variables as input and produces their Boolean sum as output.

★ The same operation holds good for an AND gate with more than two inputs.



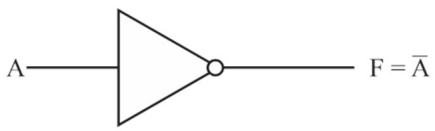
Symbol for an AND gate

Input		Output
A	B	F = A . B
1	1	1
1	0	0
0	1	0
0	0	0

NOT gate-

★ A NOT gate has only one input and one output.

★ It is also called an inverter. Its output is the complement of the input.

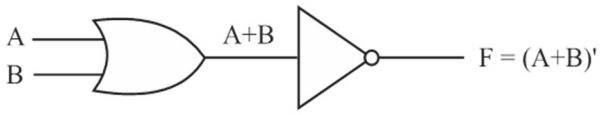


A	F = A'
1	**0**
0	**1**

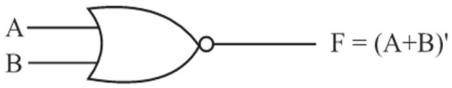
NOR gate-

★ This gate is logically equivalent to a NOT gate with an OR gate.

★ The NOR function is the complement of the OR function.



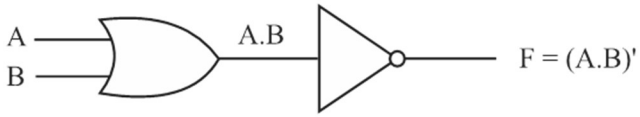
A NOT gate in series with an OR gate.



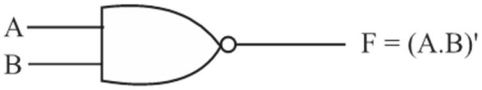
A	B	F = (A + B)'
1	1	0
1	0	0
0	1	0
0	0	1

NAND gate-

★ This gate is logically equivalent to a NOT gate in series with an AND gate.



A NOT gate series with an AND gate



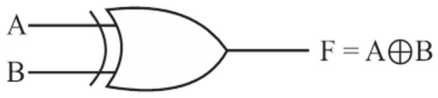
A	B	F = (A.B)'
1	1	0
1	0	1
0	1	1
0	0	1

Exclusive OR gate-

★ The full name of the OR gate is the inclusive OR gate.

★ The exclusive OR gate is different. It gives an output of 1 when either but not both inputs are I.

★ In Boolean algebra, the sign stands for the XOR operation.



Symbol for a XOR gate

Input A	Input B	Output F = A'B + AB'
0	0	0
0	1	1
1	0	1
1	1	0

Exclusive – NOR (XNOR) gate-

★ An XNOR gate is equivalent to an Exclusive OR gate followed by an inverter.

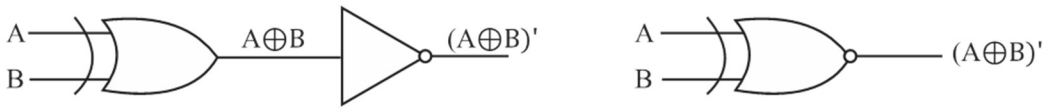
★ The output is I only we both inputs are either 0 or 1.

Input		**Output**
A	**B**	**F = AB + A'B'**
0	0	1
0	1	0
1	0	0
1	1	1

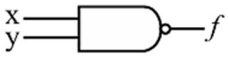
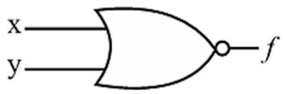
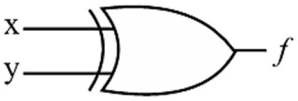
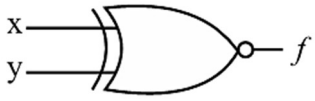
The XNOR operation can be written as follows:

★ F= AB + A' B'
= A XNOR B
= (A⊕B)'

★ The logic diagram and logic symbol are shown in the figure.

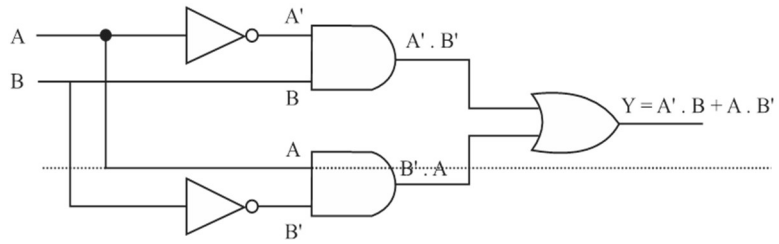


Logic gate with Truth Table:

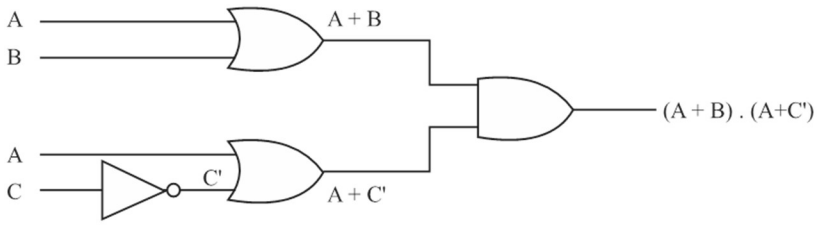
Name	Symbol	Truth Table	Boolean Formula
AND		**x** **y** **f** 0 0 0 0 1 0 1 0 0 1 1 1	**f = xy**
OR		**x** **y** **f** 0 0 0 0 1 1 1 0 1 1 1 1	**f = x + y**
NOT (inverter)		**x** **f** 0 1 1 0	**f = x`**
NAND		**x** **y** **f** 0 0 1 0 1 1 1 0 1 1 1 0	**f = (xy)`**
NOR		**x** **y** **f** 0 0 1 0 1 0 1 0 1 1 1 0	**f = (x + y)`**
NOR (Exclusive-OR)		**x** **y** **f** 0 0 0 0 1 1 1 0 1 1 1 0	**f = x⊕y = x`y + xy`**
XNOR (Exclusive-NOR)		**x** **y** **f** 0 0 1 0 1 0 1 0 0 1 1 1	**f = (x⊕y)` = xy + x`y`**

Interconnecting Gates:

1. **Meaning:**
   - Logic gates can be **connected together** to form **gating networks** or **logic networks** that perform more complex logical operations.
2. **Deriving Boolean Expression:**
   - The **Boolean expression** for a network can be found by **progressing systematically** from the **inputs** toward the **final output**, analyzing each gate step by step.
3. **Shared Inputs:**
   - In a logic network, some gates may **share the same input signals**.
4. **Methods of Representation:**
   - **Method 1:** Indicate **inputs separately** for each gate.
   - **Method 2:** Use **branching lines** to show that a **single input** is connected to **multiple gates**.
5. **Purpose:**
   - Interconnecting gates allows designers to **implement complex Boolean functions** and **build digital systems** like adders, multiplexers, and control circuits.



(a)



(b)

बूलियन बीजगणित और तर्क द्वार: भाग-1

विषय-वस्तु:

बूलीय बीजगणित के स्वयंसिद्ध, कुछ उपयोगी प्रमेय, अद्वितीय विशेषताएँ, बूलीय उपबीजगणित, मूलभूत संक्रियाएँ, तर्क गेट।

बूलीय बीजगणित की परिभाषा:

- बूलीय बीजगणित एक अरिक्त समुच्चय है जिसमें संक्रियाएँ + , . , ‘और स्थिरांक 0 और 1 कुछ नियमों का पालन करते हैं।

बूलीय बीजगणित के स्वयंसिद्ध:

- यदि $a, b, c \in B$
- क्रमविनिमेय नियम
  - $a + b = b + a$
  - $a \cdot b = b \cdot a$
- वितरण नियम
  - $a + (b \cdot c) = (a + b)(a + c)$
  - $a \cdot (b + c) = (a \cdot b) + (a \cdot c)$
- तत्समक नियम
  - $a + 0 = a$
  - $a \cdot 1 = a$
- पूरक नियम
  - $a + a' = 1$
  - $a \cdot a' = 0$

मूल प्रमेय

- शून्यभावी नियम:
  - $a + a = a$
  - $a \cdot a = a$
- परिबद्धता नियम:
  - $a + 1 = 1$
  - $a \cdot 0 = 0$
- अवशोषण नियम:
  - $a + (a \cdot b) = a$
  - $a(a + b) = a$
- सहचारी नियम:
  - $(a + b) + c = a + (b + c)$
  - $(a \cdot b) \cdot c = a \cdot (b \cdot c)$
- पूरक की विशिष्टता:
  - यदि $a + x = 1$ AND $a \cdot x = 0 \rightarrow x = a'$
- अन्तर्वलन-नियम:
  - $(a')' = a$
  - $0' = 1$
  - $1' = 0$

- डी-मॉर्गन का नियम:
  - $(a + b)' = a'b'$
  - $(ab)' = a' + b'$
- जब संदर्भ से स्पष्ट हो, तो हम बूलीय बीजगणित में बिंदु (·) चिह्न को हटा सकते हैं।
  उदाहरण: $a \cdot b$ लिखने के बजाय, हम सीधे ab लिख सकते हैं।
  इसकी अनुमति तभी दी जाती है जब गलतफहमी की कोई संभावना न हो।
- इस पाठ में कुछ बूलीय नियमों को बीजगणितीय प्रमाणों के साथ दर्शाया गया है।
- शेष नियमों को गृहकार्य या अभ्यास के रूप में दिया जाता है ताकि शिक्षार्थी उन्हें स्वयं सिद्ध कर सकें।
- **स्मरणीय बिंदु:**
  - सत्य सारणी का उपयोग करके भी प्रत्येक नियम को सिद्ध किया जा सकता है।
  - प्रमेय: $a + a = a$
  - प्रमेय: $a.a = a$
  - प्रमेय: $a + 1 = 1$
  - प्रमेय: $a + (a.b) = a$
  - प्रमेय: प्रत्येक $a \in B$ के लिए, a' अद्वितीय है।
  - प्रमेय: तत्व 0 और 1 अद्वितीय हैं
  - प्रमेय: (i) $(a+b)' = a'b'$ (ii) $(a.b)' = a' + b'$

बूलीय बीजगणित की विशिष्ट विशेषताएँ:

- केवल दो मान: 0 और 1
  - बूलीय बीजगणित में केवल 0 (गलत) और 1 (सही) का उपयोग होता है।
  - साधारण बीजगणित में अनंत मानों (1, 2, 3, ... आदि) का उपयोग होता है।
  - इसलिए, प्रत्येक बूलीय चर के केवल दो ही संभावित मान हो सकते हैं: 0 या 1।
- घटाव या भाग नहीं।
  - बूलीय बीजगणित केवल AND, OR और NOT संक्रियाओं का समर्थन करता है।
  - इसमें सामान्य बीजगणित की तरह घटाव (−) या भाग (÷) नहीं होता है।
- पूरक (NOT) संक्रिया अद्वितीय है।
  - बूलीय बीजगणित में पूरक (A') नामक एक विशेष एकात्मक संक्रिया होती है।
  - सामान्य बीजगणित में इस बार/NOT संक्रिया का कोई समतुल्य नहीं है।

  बूलीय उप-बीजगणित:
  - एक बूलीय उप-बीजगणित, एक बूलीय बीजगणित का एक उपसमुच्चय होता है।
  - यह उपसमुच्चय रिक्त नहीं होना चाहिए।
  - इसे उप-बीजगणित केवल तभी कहा जाता है जब यह उन्हीं संक्रियाओं का पालन करता है:
    OR (+)
    AND (·)
    NOT (')
    और इसमें **0** और **1** शामिल हों।
- **अर्थ:**
  - यदि a और b, A में हैं → तो $a + b$, $a \cdot b$ और a' भी A में होने चाहिए।
  - उदाहरण समुच्चय (D_{30})
    30 के सभी विभाजकों के समुच्चय पर विचार करें:
    $D_{30} = \{1, 2, 3, 5, 6, 10, 15, 30\}$

- D_{30} पर परिभाषित संक्रियाएँ
  - a + b = lcm(a, b) (जहाँ + का अर्थ LCM है, सामान्य जोड़ नहीं)
  - a · b = gcd(a, b) (जहाँ · का अर्थ GCD है, गुणन नहीं)
  - a' = 30 / a (पूरक को 30 को उस संख्या से भाग देने पर प्राप्त मान के रूप में परिभाषित किया जाता है।)
- D_{30} एक बूलीय बीजगणित है
  - पहले परिभाषित की गई संक्रियाओं (LCM, GCD, पूरक) के साथ,
  - D_{30} एक वैध बूलीय बीजगणित बन जाता है।

    D_{30} के बूलीय उपबीजगणित के उदाहरण

    $A_1 = \{1, 5, 6, 30\}$

    $A_3 = \{1, 2, 3, 6\}$

    ये समुच्चय बूलीय उपबीजगणित हैं क्योंकि:

    वे संक्रियाओं के तहत बंद हैं:

    a + b = LCM(a, b)

    a · b = GCD(a, b)

    a' = 30/a

अर्थ: A_1 या A_3 के तत्वों पर इनमें से कोई भी संक्रिया करने पर ऐसा परिणाम प्राप्त होता है जो अभी भी उसी समुच्चय के भीतर ही रहता है।

- A_4 एक बूलीय उपबीजगणित नहीं है

  $A_4 = \{1, 2, 3, 6\}$

  जब हम 6 का पूरक लेते हैं:

  6' = 30/6 = 5

  लेकिन 5, A_4 में नहीं है, इसलिए A_4 पूरक के अंतर्गत संवृत नहीं है।

  इसलिए, A_4 एक बूलीय उपबीजगणित नहीं है।
- संवरक नियम की जाँच करें (महत्वपूर्ण बिंदु)

  किसी समुच्चय A को बूलीय उपबीजगणित सिद्ध करने के लिए, आपको सभी संक्रियाओं की जाँच करने की आवश्यकता नहीं है। इसके लिए संवरक की जाँच करना पर्याप्त है:

  {+, '} (LCM और पूरक)

  या

  {·, '} (GCD और पूरक)

  यदि इन दोनों संक्रियाओं के परिणाम A के अंदर ही रहते हैं, तो यह एक उपबीजगणित है।

मूल संक्रियाएँ:

- तीन संक्रियाएँ:
  - तार्किक योग (OR)
  - तार्किक गुणन (AND)
  - पूरकन (NOT)

तर्क 0	**असत्य**	**बंद**	**निम्न**	**खुला**	**नहीं**
तर्क 1	**सत्य**	**चालू**	**उच्च**	**बंद**	**हाँ**

- **तार्किक योग (OR):**
  - बूलीय बीजगणित में प्रत्येक चर के दो मान होते हैं: सत्य या असत्य (1 या 0), इस संक्रिया को निम्नानुसार परिभाषित किया जा सकता है:
  - **सत्य मान:**

    0 + 0 = 0 0 + 1 = 1 1 + 0 = 1 1 + 1 = 1
  - '+' तार्किक OR है (अंकगणितीय जोड़ नहीं)।

❖ सत्य सारणी: [निर्गत 1 होता है जब कम से कम एक निवेश 1 हो।]

A	B	A + B (OR)
0	0	0
0	1	1
1	0	1
1	1	1

❖ **तार्किक गुणन (AND):**

★ तार्किक गुणन का अर्थ बूलीय बीजगणित में AND संक्रिया है।

★ प्रयुक्त प्रतीक:

A · B (डॉट)

या सरल रूप में AB

या इसे A AND B के रूप में लिखा जा सकता है।

★ AND नियम:

निर्गत 1 तभी होगा जब दोनों निवेश 1 हों; अन्यथा, निर्गत 0 होगा।

★ सत्य मान:

$0 \cdot 0 = 0$ $0 \cdot 1 = 0$ $1 \cdot 0 = 0$ $1 \cdot 1 = 1$

❖ सत्य सारणी: [निर्गत 1 तभी होता है जब दोनों निवेश 1 हों।]

A	B	A. B (AND)
0	0	0
0	1	0
1	0	0
1	1	1

❖ पूरकन (NOT):

★ बूलीय बीजगणित में, पूरकीकरण एक संक्रिया है जिसे एक चर के ऊपर बार (–) या **एपोस्ट्रोफी (′)** चिह्न द्वारा दर्शाया जाता है।

प्रतीक निरूपण:

$0'=1$

$1'=0$

व्याख्या:

A′ का अर्थ **A का पूरक** है।

इसे **"NOT A"** पढ़ा जाता है।

पूरक, A का विपरीत तार्किक मान देता है

यदि $A=1$ है, तो $A'=0$ होगा।

यदि $A=0$ है, तो $A'=1$ होगा।

★ **द्वैतता सिद्धांत**

1. **परिभाषा:**

   बूलीय बीजगणित में **द्वैतता सिद्धांत** यह बताता है कि यदि कोई व्यंजक या प्रमेय सत्य है, तो उसका **द्वैत** —जो कुछ संक्रियाओं और तत्समक तत्वों को आपस में बदलकर प्राप्त किया जाता है—भी सत्य होता है।

2. **विनिमय नियम:**

   - प्रत्येक **AND (·)** को **OR (+)** से प्रतिस्थापित करें
   - प्रत्येक **OR (+)** को **AND (·)** से प्रतिस्थापित करें
   - प्रत्येक **0** को **1** से प्रतिस्थापित करें
   - प्रत्येक **1** को **0** से प्रतिस्थापित करें

3. **मुख्य विचार:**
   - संक्रियाओं और तत्समक तत्वों को बदलने का यही गुण द्वैतता को परिभाषित करता है।
   - यदि किसी प्रमेय को बूलीय बीजगणित के स्वयंसिद्धों का उपयोग करके सिद्ध किया जाता है, तो उसका द्वैत प्रमेय **स्वतः ही सत्य होता है।**
4. **स्वयंसिद्ध और प्रमेय:**
   - बूलीय बीजगणित के स्वयंसिद्धों और मूल प्रमेयों को अक्सर **युग्मों (a) और (b)** में समूहित किया जाता है, जहाँ द्वैतता के सिद्धांत को लागू करके एक से दूसरे को प्राप्त किया जा सकता है।
5. **उदाहरण:**
   - दिया गया है: $f=(a\cdot 1)\cdot(0\cdot a')$
   - द्वैत: $f^D = (a + 0) + (1 + a')$
   - यह द्वैत **AND (·) ↔ OR (+)** और **0 ↔ 1** को आपस में बदलकर प्राप्त किया जाता है।

★ **डी मॉर्गन का प्रमेय**

1. **अर्थ:**
   - डी-मॉर्गन के प्रमेय का उपयोग किसी भी बूलीय व्यंजक का **पूरक** ज्ञात करने के लिए किया जाता है।
   - यह तार्किक व्यंजकों को सरल बनाने और रूपांतरित करने में सहायक होता है।
2. **नियम:**

   डी-मॉर्गन के प्रमेय के **दो भाग** हैं:
   - $(x+y)'=x' . y'$ → एक OR का पूरक पूरकों के AND के बराबर होता है।
   - $(xy)'=x'+y'$ → AND का पूरक, पूरकों के OR के बराबर होता है।
3. **उपयोग:**
   - बड़े बूलीय व्यंजकों का पूरक ज्ञात करने के लिए इन प्रमेयों को चरण-दर-चरण लागू करें।
   - वे विशेष रूप से तर्क परिपथ को सरल बनाने और डिजिटल डिजाइन में उपयोगी होते हैं।

x	y	x'	y'	x+y	(x+y)'	x'y'	xy	(xy)'	x'+y'
0	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	0	0	1	1
1	0	0	1	1	0	0	0	1	1
1	1	0	0	1	0	0	1	0	0

विस्तारित डी मॉर्गन प्रमेय:

1. **एकाधिक चरों का विस्तार:**
   - डी मॉर्गन के प्रमेय को **दो से अधिक चरों** पर लागू किया जा सकता है।
2. **उदाहरण 1 – OR संक्रिया के लिए:**

   $(x+y+z)'=x' y' z'$
   - अतः एकाधिक चरों के OR का पूरक उनके पूरकों के **AND के बराबर होता है।**
3. **उदाहरण 2 – AND संक्रिया के लिए:**

   $(x\ y\ z)'=x'+y'+z'$
   - अतः, एकाधिक चरों के AND का पूरक उनके **पूरकों के OR के बराबर होता है।**
4. **सामान्यीकरण:**
   - इन परिणामों को इसी पैटर्न का उपयोग करके **किसी भी संख्या में चरों तक विस्तारित** किया जा सकता है।
5. **अनुप्रयोज्यता:**
   - ये प्रमेय तब भी **समान रूप से मान्य** हैं, भले ही **x और/या y जटिल बूलीय व्यंजकों** (जिनमें एक से अधिक चर हों) का प्रतिनिधित्व करते हों।

डी मॉर्गन के संयुक्त प्रमेय:

1. **उद्देश्य:**
   - डी मॉर्गन प्रमेय के दोनों रूपों को मिलाकर **किसी भी बूलीय व्यंजक का पूरक** ज्ञात करने के लिए **एक सामान्य नियम** बनाया जा सकता है।
2. **पूरक प्राप्त करने के चरण:**
   व्यंजक में निम्नलिखित को प्रतिस्थापित करें:
   (a) **0 → 1**
   (b) **1 → 0**
   (c) **+ → · (AND)**
   (d) **· → + (OR)**
   (e) **प्रत्येक चर → उसका पूरक**
3. **सामान्यीकृत सूत्र:**
   $[f(X1,X2,\ldots,Xn,0,1,+,\cdot)]'=f(X1',X2',\ldots,Xn',1,0,\cdot,+)$
   स्पष्टीकरण:
   - यह नियम किसी भी **बूलीय फंक्शन के पूरक को व्यवस्थित रूप से ज्ञात करने** में मदद करता है, चाहे वह कितना भी जटिल क्यों न हो।
   - यह डी मॉर्गन के दोनों प्रमेयों के तर्क को एक ही रूपांतरण नियम में संयोजित करता है।

तर्क गेट:

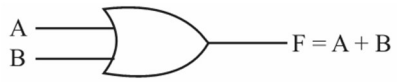
1. **परिभाषा:**
   - **तर्क गेट** एक **इलेक्ट्रॉनिक परिपथ** है जो एक या अधिक निवेश सिग्नलों पर एक विशिष्ट तार्किक क्रिया करके एक **एकल निर्गत सिग्नल** उत्पन्न करता है।
2. **प्रकृति:**
   - तर्क गेट **अंकीय परिपथ** होते हैं, जिन्हें **तर्क परिपथ** भी कहा जाता है, क्योंकि वे द्विआधारी सिग्नलों (0 और 1) के साथ कार्य करते हैं।
3. **निरूपण:**
   - परिपथ आरेखों में प्रत्येक गेट को एक **प्रतीक** द्वारा दर्शाया जाता है।
   - **निवेश लाइनें** प्रतीक में **बाईं ओर** से आती हैं।
   - **निर्गत लाइन दाईं ओर** दिखाई देती है।
4. **पूरक सिग्नल:**
   - निवेश या निर्गत लाइन पर रखा गया एक **छोटा वृत्त (बुलबुला)** उस लाइन पर सिग्नल के पूरक (व्युत्क्रमण) को दर्शाता है।
5. **उद्देश्य:**
   - तर्क गेट अंकीय इलेक्ट्रॉनिक्स के **मूलभूत निर्माण खंड** हैं और इनका उपयोग **कंप्यूटर, प्रोसेसर और तर्क-आधारित उपकरणों** में किया जाता है।

OR गेट:

1. **परिभाषा:**
   - **OR गेट** एक इलेक्ट्रॉनिक परिपथ है जो **दो या दो से अधिक निवेश सिग्नल** प्राप्त करता है और एक निर्गत उत्पन्न करता है जो निवेश का तार्किक OR होता है।
2. **तर्क फंक्शन:**
   - यदि **कम से कम एक निवेश 1 (उच्च) हो** तो निर्गत **1 (उच्च)** होगा।
   - निर्गत **0 (निम्न)** तभी होता है जब **सभी निर्गत 0 (निम्न) हों।**

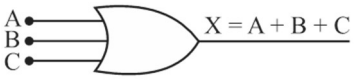
3. **निवेश अवस्था:**
   - प्रत्येक निवेश सिग्नल की **दो स्थिर अवस्थाएँ** होती हैं:
     - **1 (उच्च)** → तर्क सत्य
     - **0 (निम्न)** → तर्क असत्य
4. **प्रतीक:**
   - OR गेट के प्रतीक में **बाईं ओर** कई निवेश लाइनें होती हैं, जो एक **घुमावदार आकार** में जुड़ी होती हैं, और **दाईं ओर** एक **एकल निर्गत लाइन** होती है।
5. **सत्य सारणी (2-निवेश OR गेट के लिए):**

निवेश A	निवेश B	निर्गत (A + B)
0	0	0
0	1	1
1	0	1
1	1	1



Symbol for an OR gate

इसी अवधारणा को दो से अधिक निवेश पर भी लागू किया जा सकता है। एक तीन-निवेश वाला OR गेट और उसकी सत्य सारणी।



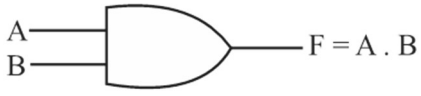
Symbol for these input OR gate

निवेश			निर्गत
A	B	C	X = A + B + C
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

AND गेट -

★ AND एंड गेट एक इलेक्ट्रॉनिक उपकरण है जो दो या दो से अधिक बूलीय चरों के मानों को निवेश के रूप में स्वीकार करता है और उनके बूलीय योग को निर्गत के रूप में उत्पन्न करता है।

★ यही संक्रिया दो से अधिक निवेश वाले AND गेट के लिए भी लागू होती है।



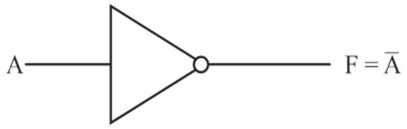
Symbol for an AND gate

Input		Output
A	B	F = A . B
1	1	1
1	0	0
0	1	0
0	0	0

NOT गेट-

★ एक NOT गेट में केवल एक निवेश और एक निर्गत होता है।

★ इसे इन्वर्टर भी कहा जाता है। इसका निर्गत निवेश का पूरक होता है।

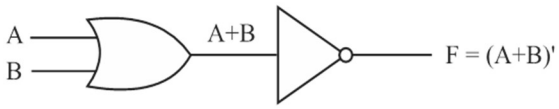


A	F = A'
1	0
0	1

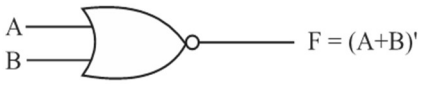
NOR गेट-

★ यह गेट तार्किक रूप से एक NOT गेट और एक OR गेट के समतुल्य है।

★ NOR फंक्शन, OR फंक्शन का पूरक है।



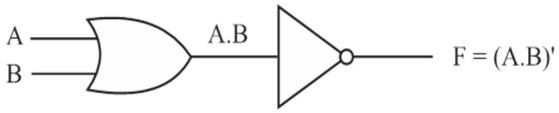
A NOT gate in series with an OR gate.



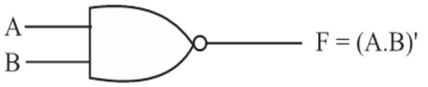
A	B	F = (A + B)'
1	1	0
1	0	0
0	1	0
0	0	1

NAND गेट-

★ यह गेट तार्किक रूप से एक NOT गेट और एक AND गेट के श्रेणीक्रम संयोजन के समतुल्य है।



A NOT gate series with an AND gate



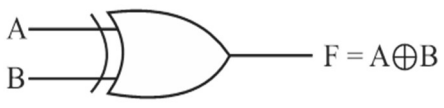
A	B	F = (A.B)'
1	1	0
1	0	1
0	1	1
0	0	1

एक्सक्लूसिव OR गेट -

★ OR गेट का पूरा नाम इनक्लूसिव OR गेट है।

★ एक्सक्लूसिव OR गेट अलग होता है। यह 1 का निर्गत तब देता है जब दोनों निवेश में से कोई एक 1 हो, लेकिन दोनों निवेश 1 नहीं होते।

★ बूलीय बीजगणित में, चिह्न XOR संक्रिया को दर्शाता है।



Symbol for a XOR gate

Input A	Input B	Output F = A'B + AB'
0	0	0
0	1	1
1	0	1
1	1	0

एक्सक्लूसिव– NOR (XNOR) गेट-

★ एक XNOR गेट, एक एक्सक्लूसिव OR गेट के समतुल्य होता है जिसके बाद एक इन्वर्टर लगा होता है।

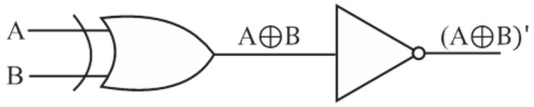
★ निर्गत केवल तभी I होता है जब दोनों निवेश या तो 0 हों या 1 हों।

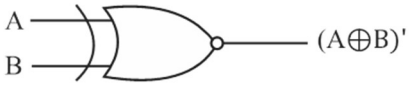
निवेश		निर्गत
A	**B**	**F = AB + A'B'**
0	0	1
0	1	0
1	0	0
1	1	1

XNOR संक्रिया को निम्न प्रकार से लिखा जा सकता है:

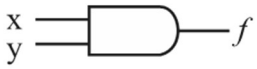
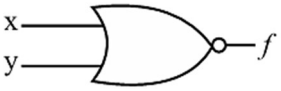
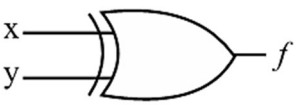
★ F= AB + A' B'
= A XNOR B
= (A⊕B)'

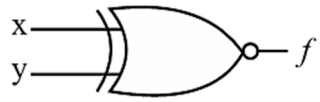
★ चित्र में तर्क आरेख और तर्क प्रतीक दर्शाए गए हैं।





सत्य सारणी सहित तर्क गेट:

नाम	संकेत	सत्य तालिका	बूलियन फॉर्मूला
AND		**x** **y** **f** 0 0 0 0 1 0 1 0 0 1 1 1	**f = xy**
OR		**x** **y** **f** 0 0 0 0 1 1 1 0 1 1 1 1	**f = x + y**
NOT (इन्वर्टर)		**x** **f** 0 1 1 0	**f = x`**
NAND		**x** **y** **f** 0 0 1 0 1 1 1 0 1 1 1 0	**f = (xy)`**
NOR		**x** **y** **f** 0 0 1 0 1 0 1 0 1 1 1 0	**f = (x + y)`**
NOR (अनन्य-OR)		**x** **y** **f** 0 0 0 0 1 1 1 0 1 1 1 0	**f = x⊕y = x`y + xy`**

XNOR (अनन्य-NOR)		**x**	**y**	**f**
		0	0	1
		0	1	0
		1	0	0
		1	1	1
				f = (x⊕y)` = xy + x`y`

अंतर संबद्ध गेट:

1. **अर्थ:**
   - तर्क गेटों को **आपस में जोड़कर गेटिंग नेटवर्क** या **तर्क नेटवर्क** बनाया जा सकता है जो अधिक जटिल तार्किक संक्रियाएँ करते हैं।
2. **बूलियन व्यंजक व्युत्पत्ति:**
   - किसी नेटवर्क के लिए **बूलियन व्यंजक** को **निवेश से अंतिम निर्गत** की ओर व्यवस्थित रूप से आगे बढ़ते हुए, प्रत्येक गेट का **चरण दर चरण** विश्लेषण करके पाया जा सकता है।
3. **साझा निवेश:**
   - एक तर्क नेटवर्क में, कुछ गेट **समान निवेश सिग्नल साझा कर सकते** हैं।
4. **निरूपण की विधियाँ:**
   - **विधि 1:** प्रत्येक गेट के लिए **निवेश अलग-अलग दर्शाए।**
   - **विधि 2: शाखाओं वाली रेखाओं** का उपयोग करके यह दर्शाएं कि **एक निवेश** कई गेटों से संबद्ध है।
5. **उद्देश्य:**
   - अंतर संबद्ध गेट डिजाइनरों को जटिल बूलीय फंक्शनों को लागू करने और योजक, बहुसंकेतक और नियंत्रण परिपथ जैसे अंकीय प्रणाली बनाने की अनुमति देते हैं।

